

K-means plus Voronoi Borders

Agnus Thomas

05/04/2026

Statement of Purpose

The statement of purpose is to be referred to from the proposal as such:

“This project explores how optimizing the placement of points - in this case, geographical points - using k-center clustering helps in creating Voronoi diagrams that estimate existing state borders, and analyzes the estimation error between the computed Voronoi diagrams and the existing state borders.”

Background

Voronoi diagrams have been used for political districting in the past since it is easier to make a diagram that is actually representative of an existing metric like population. Many Voronoi alterations exist, like Power Diagram based Voronoi and Weighted Voronoi diagrams, which can be used for many use cases. There have been case studies on how Weighted Voronoi Diagrams have been attempted to be used for political districting, and how better approaches and alterations create more accurate, legally compliant diagrams [11, 12]. However, no matter the approach, centers are the most important thing as it can determine how well a Voronoi is computed. A proposed approach utilized in this report is applying k-means to get centers, and using those k-means centers we can create a Voronoi diagram. Most work goes into how Voronoi diagrams can help k-means rather than how k-means help Voronoi diagrams, and there are proven studies that show that it improves k-means like Procedia Technology's study where D. Reddy and P. K. Jana shows it reduces error rates when it comes to recreating recognizable shapes like a heart or a triangle [8]. There has been one paper discussing how k-means can help Voronoi however, where it utilizes the MacQueen's method (which will also be used in this report) for generating k-means clusters for a voronoi diagram, which ended up having its performance affect results to the point they considered alterations [9]. Along with MacQueen's method, this paper will also utilize Additively Weighted Voronoi algorithms from Novaes et al.'s research [10].

Research Method

To test out how well k-means can help Voronoi diagrams, Python libraries will be used as well as Geojson files. There are four kinds of tests that occurred: Using the original Voronoi, the k-means Voronoi, Weighted Voronoi with k-means and finally, k-means but with the MacQueen's method. This will be tested on the map of North Carolina to attempt to replicate its counties and the mainland US for its states. Each test will give a plot of the Voronoi diagram with centers highlighted, and will give descriptive statistics on how much of the diagram is away from the border within 1 km as well as the mean, median and max distances of the produced diagrams. Each test will be run three times since it has been noticed that results change. While the original Voronoi does not have such changes occurring, three runs have been done for fairness sake.

This project utilizes the following libraries:

- GeoPandas: An open source project focused on dealing with geospatial data and provides important data structures like GeoDataFrame which is used frequently in this project. [7]
- SciPy: Provides the original Voronoi algorithm and k-means method. [6]
- Numpy, Pandas: Numpy provides a lot of mathematical operations used frequently in implementations and throughout the produced scripts for the tests as well, while pandas gives functions for GeoPandas [3,5].
- Shapely: Utilized by GeoPandas for geometrical purposes and as well as used frequently in implementations [4].
- Matplotlib: This is used for plotting diagrams [14].

The data comes from the following sources:

- The US Census Bureau has a directory for the needed shapefiles for mainland US and North Carolina [1].
- Mapbox allows users to mark points on a globe which can be downloaded to a Geojson format which is what was used as points [2].

The Projections used are as such:

- EPSG:32119: This one is for North Carolina [13].
- EPSG:5070: This one focuses more on Canada and the US [13].

Originally, EPSG:4326 was used since it was a more neutral projection, however, this proved to be inaccurate when calculating statistics. In addition, it was better to convert it to a more accurate system for each map rather than using one system since one projection could not be suitable for certain maps and places.

Implementations

This section will go through all the implementations of each method:

- The original Voronoi uses an implementation from SciPy which uses Qhull libraries. It takes a dual of Delaunay Triangulation [15].
- The k-means method takes an implementation from SciPy as well. The points are classified into clusters and the centroids are updated until the positions are stable over a certain amount of iterations which is based on comparing the absolute value of change with Euclidean distance [6].
- The Weighted Voronoi method uses an Additively Weighted Voronoi using the implementation mentioned by Novaes et al.'s paper on Voronoi algorithms [10]. Multiplicatively Weighted diagrams tend to have holes which are not sufficient for the tests, since the maps tested on do not have any exclave or enclaves, nor is the design of the possible produced diagrams suitable for districting borders. In this implementation, weights are defaulted to zero and will not be using the iterative approach from the paper to save computational time. Power Diagram Voronoi was originally going to be implemented as well but considering the amount of time to implement the Additively Weighted Voronoi method, this is better reserved for the future. A lot of the implementation was taken from methods from the Numpy library and Shapely [4,5].
- The MacQueen's method for k-means mentioned in Q. Du and T.-W. Wong's paper is described as such:
 - The input consists of a closed set, the amount of clusters, the amount of generators, probability, and a probability distribution. Output is a CVT [9].
 - Generate an array full of ones by the amount of clusters and generate random points to the generators using the probabilistic distribution. Go through a certain amount of interactions where a random sample of the closed set is picked, and the closest generator gets updated based on that random sample. Keep going until it is stable [9].

This paper utilizes the MacQueen's k-means method from the paper, however a uniform distribution was used instead of a probabilistic method ($p=1$) so most of the generated points were from the closed set of geographical points themselves. The amount of iterations was set to 500 to save computational time.

Results

North Carolina Test Run

A test run on geographical points in North Carolina was done to ensure the implementations were working properly by checking how well it matched county borders. The points picked ($k=67$) were of cities. Each implementation had 3 runs done, with all methods except the original Voronoi having 3 runs based on $k = [10, 50, 67]$ clusters. Tables 1-5 gives results of these runs and Figures 1-3 give results of the plots from the third runs and where $k=50$.

Statistics

	Original	K-means ($k=50$)	Weighted Voronoi + Scipy Keans ($k=50$)	MacQueen's Method ($k=50$)
Percentage of generated borders within 1km in distance	34.2	35.06666667	10.13333333	33.13333333
Mean	3595.481767	3847.208664	22514.87265	4039.698938
Median	2382.966366	2477.004292	7038.239805	2586.66142
Max	19444.31579	20403.24324	144957.8228	22276.47078

Table 1: Total results taken from the average of the 3 runs

	1	2	3	Average
Percentage of generated borders within 1km in distance	34.2	34.2	34.2	34.2
Mean	3595.481767	3595.481767	3595.481767	3595.481767
Median	2382.966366	2382.966366	2382.966366	2382.966366
Max	19444.31579	19444.31579	19444.31579	19444.31579

Table 2: The Original Voronoi Diagram runs and average taken from runs where $k=67$

$k=10$	1	2	3	Average
--------	---	---	---	---------

Percentage of generated borders within 1km in distance	33.6	36.2	32.6	34.13333333
Mean	4625.882541	4093.675243	4462.372652	4393.976812
Median	3187.234707	3048.200847	3479.78053	3238.405361
Max	21477.79468	18567.48354	17805.63718	19283.63847
k=50	1	2	3	Average
Percentage of generated borders within 1km in distance	34.4	36	34.8	35.06666667
Mean	3880.083237	3632.106459	4029.436296	3847.208664
Median	2415.935344	2252.508163	2762.569368	2477.004292
Max	19358.06254	21105.8986	20745.76858	20403.24324
k=67	1	2	3	Average
Percentage of generated borders within 1km in distance	35.2	34	35	34.73333333
Mean	3620.705178	3600.15197	3554.121654	3591.659601
Median	2401.914998	2375.347397	2314.644265	2363.968887
Max	19392.30647	18889.36498	19876.2869	19385.98612

Table 3: The k-means diagram runs and average for k=[10,50,67]

k=10	1	2	3	Average
Percentage of generated borders within 1km in distance	6.8	7	6.2	6.66666667
Mean	29307.51136	29942.42442	29891.07826	29713.67135
Median	8784.906806	10179.24952	9508.281328	9490.812551
Max	145649.7542	145447.0729	145314.4398	145470.4223
k=50	1	2	3	Average
Percentage of generated borders within 1km in distance	8.6	11.8	10	10.13333333

Mean	22444.18486	22567.55208	22532.88102	22514.87265
Median	7101.719437	7130.540264	6882.459713	7038.239805
Max	144924.427	145199.5036	144749.538	144957.8228
k=67	1	2	3	Average
Percentage of generated borders within 1km in distance	10.4	11.8	13.2	11.8
Mean	21236.47014	20959.15052	21389.97072	21195.19713
Median	6167.328501	6073.39376	5960.815946	6067.179402
Max	145398.1485	145021.9786	145316.5072	145245.5448

Table 4: The Weighted Voronoi diagram runs and average for k=[10,50,67]

k=10	1	2	3	Average
Percentage of generated borders within 1km in distance	32	37.6	46.2	38.6
Mean	4340.578637	4168.090516	3604.927982	4037.865712
Median	3089.716137	2504.029998	1916.393996	2503.380044
Max	18197.62832	19134.64786	17444.18198	18258.81939
k=50	1	2	3	Average
Percentage of generated borders within 1km in distance	31.4	36	32	33.13333333
Mean	4296.003082	3779.653732	4043.44	4039.698938
Median	3069.660137	2070.571651	2619.752471	2586.66142
Max	25954.8261	19229.34651	21645.23972	22276.47078
k=67	1	2	3	Average
Percentage of generated borders within 1km in distance	32.6	35	34	33.86666667
Mean	3565.585709	3565.638187	3653.519975	3594.914624
Median	2483.629545	2300.086651	2526.797802	2436.837999

Max	18071.20118	19503.4684	19716.41544	19097.02834
-----	-------------	------------	-------------	-------------

Table 5: The MacQueen's method runs and average for k=[10,50,67]

Table 1 shows that the k-means diagram shows the best result when it comes to how close it is to original borders (35.1%) followed by the original Voronoi (34.2%). The Weighted Voronoi performed the worst (10.1%) however it is possible the implementation itself may have issues since there was no control group to compare this to which can be done in a future work. It also was a considerable outlier when it came to mean, median and the max distance as well.

It is to be noted from Table 2 that when max points are used, it is most similar to the original Voronoi diagram and performed best when we used the maximum amount of points. The maximum number of clusters also produced the least highest mean, median and max when compared to k=50 which fit the boundaries the best. The k-means method (k=67, 34.7%) performed marginally better than the original Voronoi (34.2%). Table 3 shows that for Weighted Voronoi the maximum amount of clusters worked the best (11.8%) and showed the fastest improvement(k=10, 6.67%< k=50, 10.13%< k=67, 11.8%). This could mean two things: either this was not implemented well or the amount of points was not enough. This is more evident with the median having a drop in distances (k=10, 9490.812551> k=50, 7038.239805 > k=67, 6067.179402). Finally, MacQueen shows that the least amount of clusters actually performed the best, with k=10 giving 38.6% fit.

Diagrams

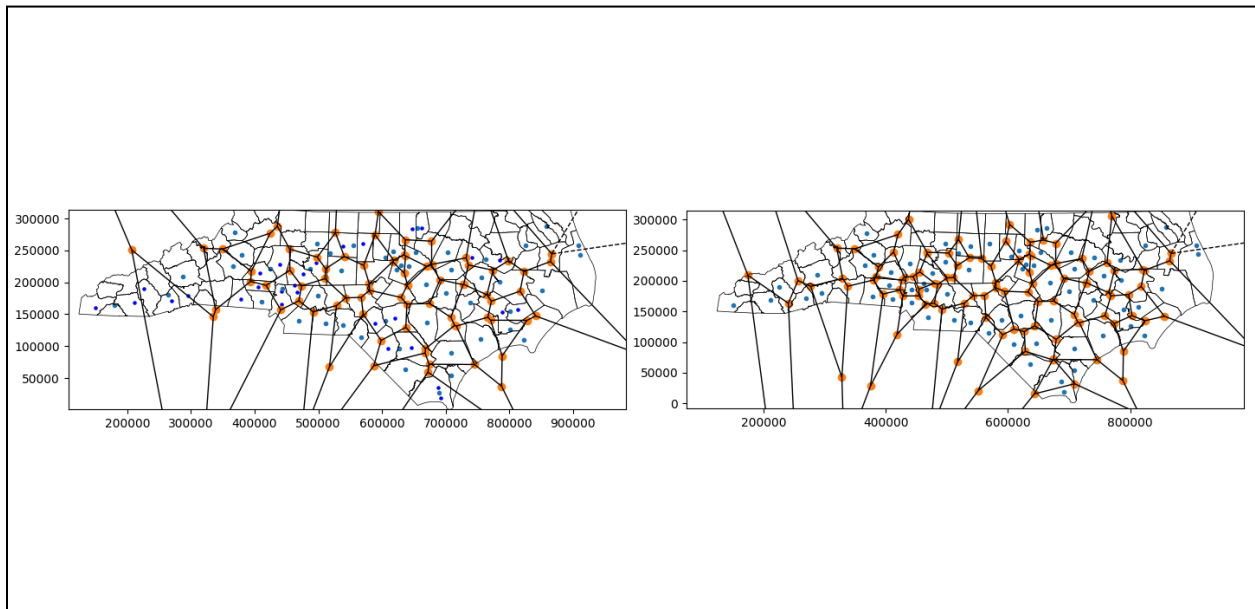


Figure 1: Left: K-means diagram Right: Original Voronoi

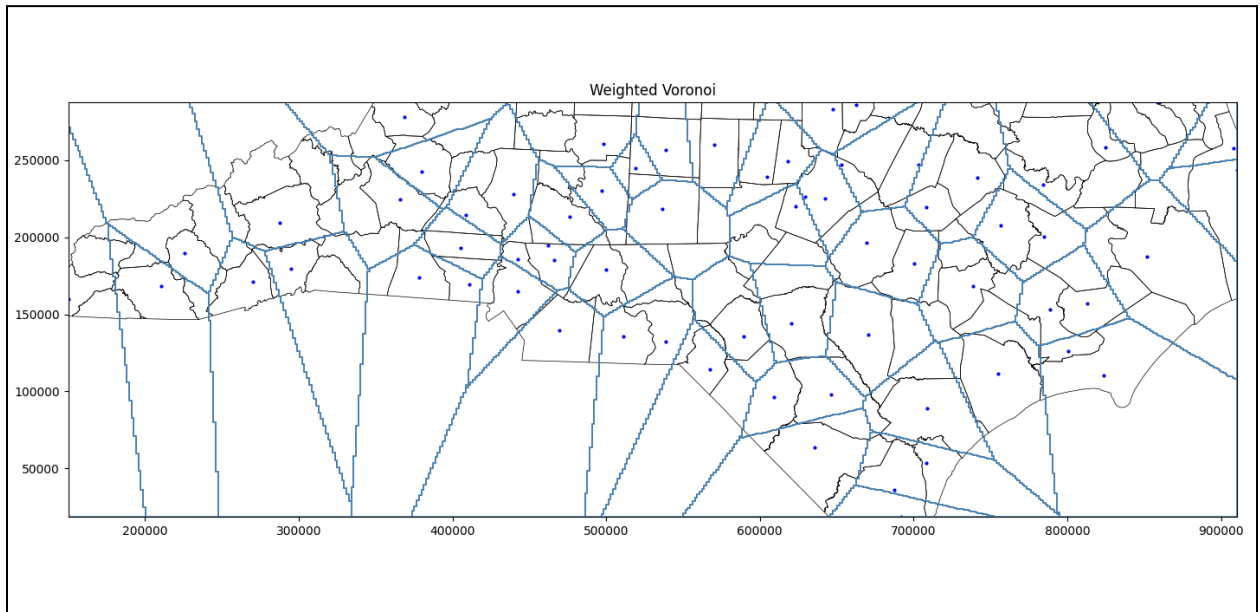


Figure 2: Weighted Voronoi Diagram

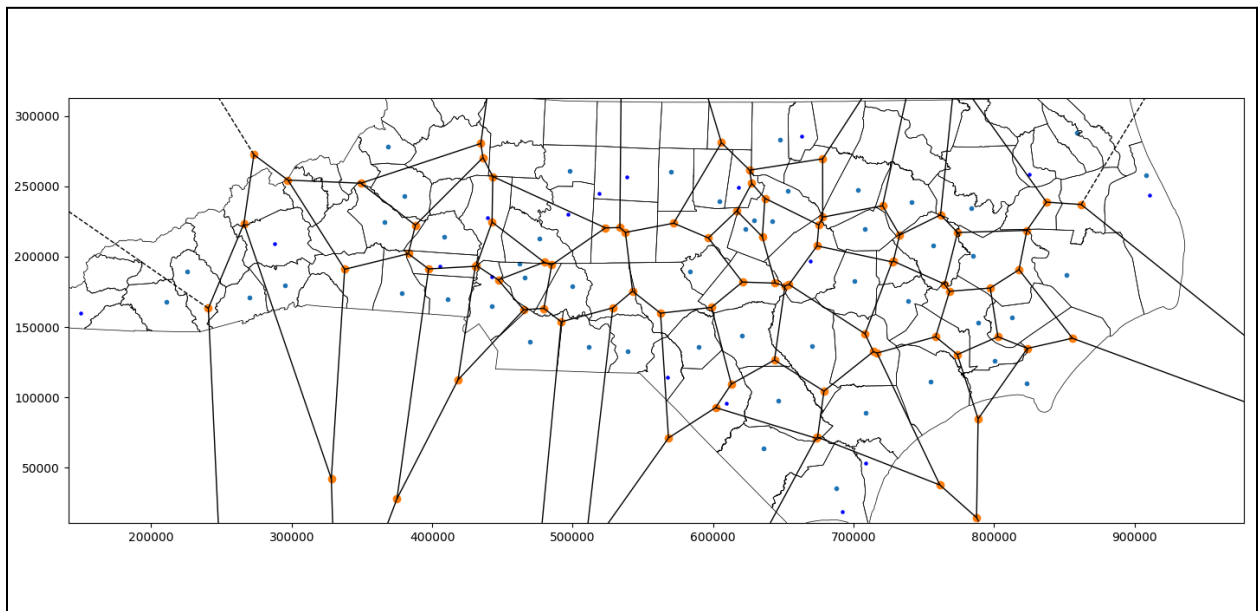


Figure 3: MacQueen's k-means diagram

These diagrams were taken where $k=50$ and at the 3rd run. The original and k-means Voronoi look considerably similar here. The original Voronoi diagram shows more concentrations in regions around the middle, while the k-means has regions more spaced out. The Weighted and MacQueen's method diagrams show a considerable amount of zoom in, which could mean that it ended up being too

compact than expected and causing certain existing regions to not be considered. All diagrams seem to struggle around the left where the state has a wedged like shape, and the diagrams themselves tend to want to spread out more.

US Mainland

A test was done on the US mainland. Only 48 states were considered as a result, while the rest and any other U.S. owned territories were excluded for the sake of simplicity. The points picked (k=291) were also from cities. Each implementation had 3 runs done, and the amount of clusters were set to 48 to get a more accurate result based on the amount of states on the mainland. Tables 6-10 gives results of these runs and Figures 4-6 give results of the plots from the third runs.

	Original	K-means	Weighted + Scipy Keans	MacQueen's Method
Percentage of generated borders within 1km in distance	16.2	27	0.8	23.66666667
Mean	55223.24586	46098.14406	127624.9596	48277.25356
Median	41499.21759	28646.37654	72366.27174	32024.73862
Max	286888.4233	284343.0667	897140.0603	269572.4968

Table 6: Total results taken from the average of the 3 runs for the US

	1	2	3	Average
Percentage of generated borders within 1km in distance	16.2	16.2	16.2	16.2
Mean	55223.24586	55223.24586	55223.24586	55223.24586
Median	41499.21759	41499.21759	41499.21759	41499.21759
Max	286888.4233	286888.4233	286888.4233	286888.4233

Table 7: The Original Voronoi Diagram runs and average taken from runs

	1	2	3	Average
Percentage of generated borders	24.8	26.2	30	27

within 1km in distance				
Mean	47009.88623	47749.20426	43535.34168	46098.14406
Median	30600.92372	30485.84299	24852.3629	28646.37654
Max	307186.491	264401.7792	281440.9298	284343.0667

Table 8: The k-means diagram runs and average for the US

	1	2	3	Average
Percentage of generated borders within 1km in distance	1.2	0.8	0.4	0.8
Mean	124473.0227	130194.1582	128207.6978	127624.9596
Median	74288.26592	71195.85579	71614.69352	72366.27174
Max	890764.9436	897421.6416	903233.5956	897140.0603

Table 9: The Weighted Voronoi runs and average for the US

	1	2	3	Average
Percentage of generated borders within 1km in distance	23.4	21	26.6	23.66666667
Mean	46469.53985	51821.2889	46540.93193	48277.25356
Median	31897.22526	36912.41225	27264.57836	32024.73862
Max	228602.4138	289363.2135	290751.863	269572.4968

Table 10: The MacQueen k-means runs and average for the US

Similarly to North Carolina, the k-means method, as seen in Table 6, worked the best (27%). More interestingly, the MacQueen's method works well here as well with the second best (23.667%), which is evident considering it worked well the less clusters that existed, and the ratio of clusters and cities being more favorable too (48:291). Weighted Voronoi still performed poorly.

Table 7 shows that the opposite might be the case as it performed poorly when compared to North Carolina's results in Table 2, as the clusters were smaller in

comparison to the amount of cities ($16.2\% < 34.2\%$). Table 8 and Table 10 show consistency when it comes to the stats from the runs, however Table 9 shows a sharp decrease from the first run to the latter two runs ($1.2\% > 0.8\% > 0.4\%$) despite sticking to the same amount of clusters used.

Diagrams

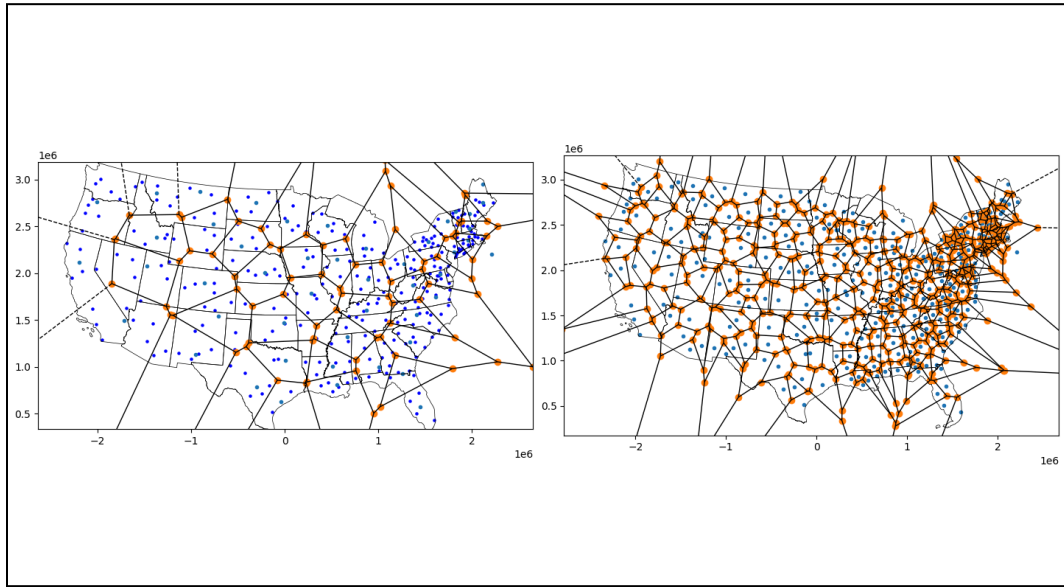


Figure 4: Left: K-means diagram Right: Original Voronoi

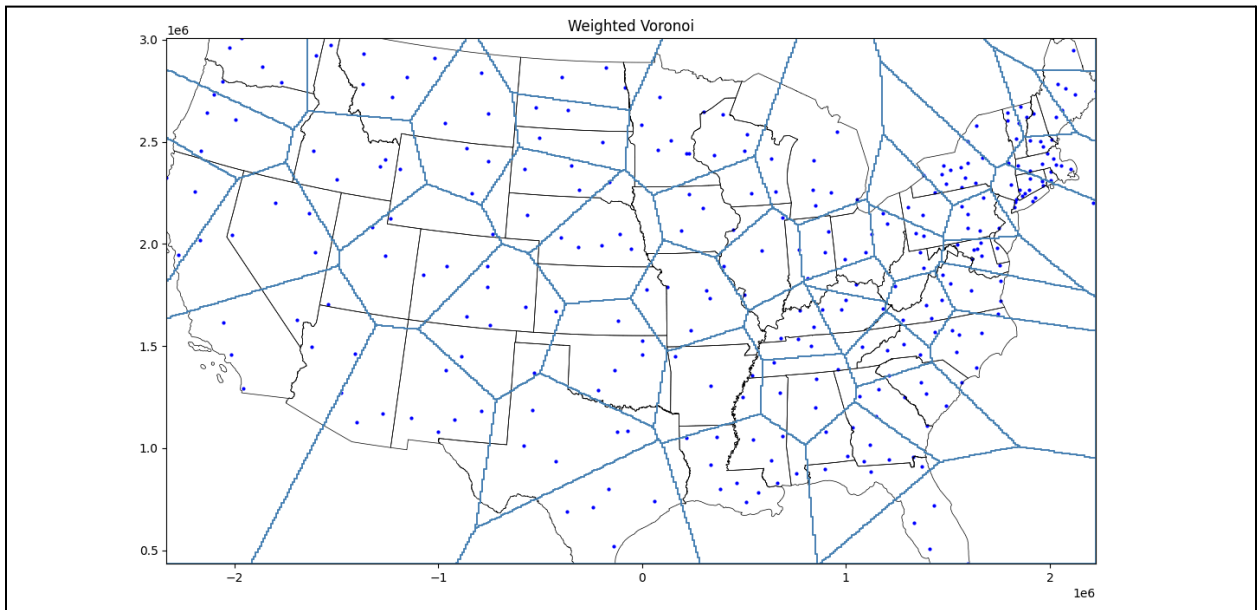


Figure 5: Weighted Voronoi Diagram

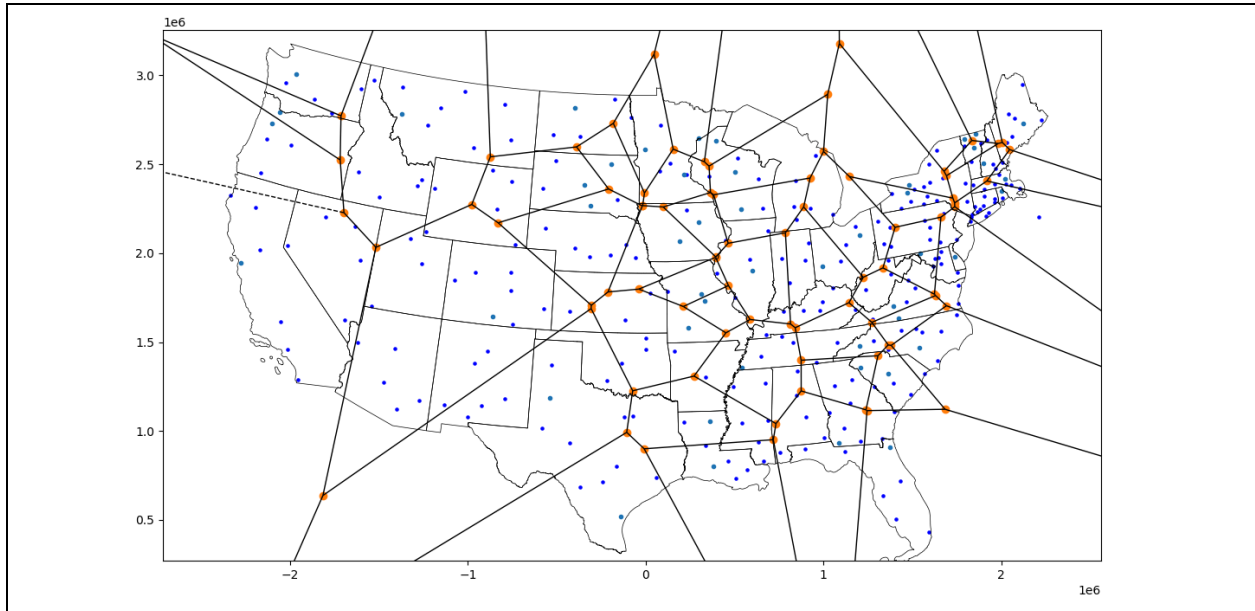


Figure 6: MacQueen's k-means diagram

Figure 4 shows a more contrasting story than the one in Figure 1. The K-means diagram did a better job giving a more consistent layout while the original, while looking visually better, has regions around the cities making it too congested. The original Voronoi however did a better job at more wedged shaped areas like in Florida and Texas, while in the k-means diagram it mostly ignored the nuances. Figure 5 also shows the same issues of Figure 2 where it has ignored the majority of some parts of the map like Washington, Florida, Texas and Maine for example. Both k-means and the Weighted-Voronoi seemed to also have the issue where it mostly makes regions where it overlaps with multiple states. This is more evident in states with squarish borders. The MacQueen's k-mean diagram in Figure 6 was the most shocking, considering it visually looks worse than the rest combined, yet performed better. Around the Illinois, Minnesota, Iowa and Missouri borders, the diagram does mostly follow the lines which could have contributed with its score. However, the west of that seemed to have performed even worse, which could indicate that it does not work well with more squarish regions.

Conclusions from the results

In conclusion, it can be considered that the k-means + original Voronoi implementation works best when it comes to creating diagrams that closely mimic political borders. While none of the runs had anything above 40%, it still gives a decent starting point into further research. Other conclusions can be derived as

well, such as MacQueen's method working better with less clusters and more points and original Voronoi diagrams like more clusters and more points.

Promising Approaches for ongoing work

Given more time, the following could be looked into considering a lot of the planned work was cut due to time constraints and the amount of time it took to implement functions:

- Using another country like Canada and Australia: The Voronoi diagrams did struggle when it came to larger regions, so trying to test on these countries could give more ideas as to why this is happening. This was planned as another test run, however it was somewhat difficult to find suitable files in the timeframe. Additionally, the rest of the US would also be promising to test to see how well it can deal with further regions.
- Verifying the Weighted Voronoi diagram itself: In hindsight, this should have had a control group run on its own considering this performed the worst out of any other method. The iterative approach was probably better to use on the weights than setting it to zero, which could have made a more accurate diagram.
- Power Voronoi diagrams: This is also a planned implementation as it seemed promising from the related work.
- Possible metrics outside of cities: When choosing a metric, cities were a decent metric considering population could help when districting. However, it is possible that other metrics like rivers would have performed better.
- A more streamlined way to process multiple runs to be more efficient while testing.

Remaining questions I want answered

The following outlines questions that had risen from working on this project:

- Was the implementation I tried to use for Weighted Voronoi inaccurate?
- Would a Compound Additive and Multiplicative Weighted voronoi have worked better? Originally, I did not implement this but considering the Weighted Voronoi struggled with the more wedged shaped regions this could have solved some issues.

Ideas that looked promising but weren't

The most promising idea was the Weighted Voronoi. However, it did not work as expected, considering it was the worst performing implementation. Additionally, MacQueen's method seemed somewhat reliable, but it did not recognize many states in the western part of the US. Another change was the projection in of itself, as the original projection EPSG:4326 was actually not a projection but it was more or so a grid like structure more or so used in GPS which used degrees [13]. A distance approach was more accurate and reasonable in the end.

Repository

The following is the repository of the code:

<https://github.com/raspberryhelp/kmeansvoronoi-cs6319>

References

- [1] "United States Census Bureau," Index of /, <https://www2.census.gov/> (accessed May 3, 2026).
- [2] Mapbox, "Powered by Mapbox," geojson.io, <https://geojson.io/> (accessed May 3, 2026).
- [3] "Pandas," pandas, <https://pandas.pydata.org/> (accessed May 3, 2026).
- [4] Shapely, <https://shapely.readthedocs.io/en/stable/#> (accessed May 3, 2026).
- [5] "NumPy documentation#," NumPy documentation - NumPy v2.4 Manual, <https://numpy.org/doc/stable/index.html> (accessed May 3, 2026).
- [6] "Scipy," SciPy, <https://scipy.org/> (accessed May 3, 2026).
- [7] "Geopandas 1.1.3#," GeoPandas 1.1.3 - GeoPandas 1.1.3+0.gf5fe3ff.dirty documentation, <https://geopandas.org/en/stable/> (accessed May 3, 2026).
- [8] D. Reddy and P. K. Jana, "Initialization for K-means clustering using Voronoi diagram," *Procedia Technology*, vol. 4, pp. 395–400, 2012.
doi:10.1016/j.protcy.2012.05.061

- [9] Q. Du and T.-W. Wong, "Numerical studies of macqueen's K-means algorithm for computing the centroidal Voronoi tessellations," *Computers & Mathematics with Applications*, vol. 44, no. 3-4, pp. 511-523, Aug. 2002.
doi:10.1016/s0898-1221(02)00165-7
- [10] A. G. N. Novaes, J. E. Souza de Cursi, A. C. L. da Silva, and J. C. Souza, "Solving continuous location-districting problems with Voronoi diagrams," *Computers & Operations Research*, vol. 36, no. 1, pp. 40-59, Jan. 2009.
doi:10.1016/j.cor.2007.07.004
- [11] F. Ricca, A. Scozzari, and B. Simeone, "Weighted voronoi region algorithms for political districting," *Mathematical and Computer Modelling*, vol. 48, no. 9-10, pp. 1468-1477, Nov. 2008. doi:10.1016/j.mcm.2008.05.041
- [12] F. Karimi, M. R. Delavar, and M. A. Mostafavi, "Political districting using Geospatial Information System," Welcome to the Frontpage,
<https://www.iadisportal.org/digital-library/political-districting-using-geospatial-information-system> (accessed Mar. 14, 2026).
- [13] K. T. G. (<https://www.klokantech.com/>), "Coordinate Systems Worldwide," EPSG.io, <https://epsg.io/> (accessed May 4, 2026).
- [14] "Visualization with python," Matplotlib, <https://matplotlib.org/> (accessed May 4, 2026).
- [15] "Qhull," Qhull code for Convex Hull, Delaunay Triangulation, Voronoi Diagram, and Halfspace Intersection about a Point, <http://www.qhull.org/> (accessed May 4, 2026).